



MEMBERS

Name	SAP ID
Ayush Negi	590015410
Ayush Kumar	590014539
Aditi Vijay	590018228
Mayank Tanishk	590015863
Mayank Parmar	590017039
Sakshi Singh	590012402
Samiksha Rajan	590016234
Satyam Pandey	590016569



INDEX

Section	Content Description	Page No.
1. Goal of the Phase	Turning the use cases from Phase 1 into a clean class design and package structure	3
2. Package Structure Design	Establishing the practical layout ensuring separation between UI, service, DAO, and model layers (model, service, dao, ui, util).	3 - 4
3. Class Responsibilities	Defining the distinct roles for early classes like Company, Student, Drive, Application, Interview Slot, DriveService, Company Dao, and Application Dao.	4 - 5
4. UML Class Relationships	Mapping the 1-to-1 and 1-to-Many relationships for the UML class diagram and establishing layer dependencies.	5 - 6
5. Design Warnings Adherence	Ensuring no giant manager classes, no SQL inside Swing event handlers, no UI-only validation, and no classes without real responsibility.	6 - 7
6. Review Checklist	Verifying that every class has a clear role , relationships are sensible , DAO/Service layers are separated, and the UML supports Phase 1 use cases.	8

Object Oriented Programming (CSEG1044)

Phase 2: UML and Package Design

Project Title: Placement Drive Management System.

- **Goal of This Phase:-**

- The primary goal of this phase is to systematically turn the use cases defined in Phase 1 into a clean class design and a robust package structure. This ensures that our application is modular, maintainable, and adheres to standard Object-Oriented Programming principles.

- **Package Structure Design**

- **To maintain a clear separation between UI, service, DAO, and model classes , the project is organized into the following practical package layout:**

- **model:** Contains all the core entity classes (Plain Old Java Objects or POJOs). These classes represent the data structures of the application and have no knowledge of the database or the UI.
- **dao (Data Access Object):** Contains classes exclusively responsible for communicating with the database. All SQL queries (INSERT, SELECT, UPDATE, DELETE) reside strictly in this package.
- **service:** Contains the core business logic and validation rules. It acts as a bridge between the ui and dao layers.

- **ui:** Contains all the Java Swing components (Frames, Panels, Dialogs, Event Listeners) that the user interacts with.
- **util:** Contains global utility classes, such as the database connection manager.
- **Class Responsibilities**
- **Based on the guidelines, here is the short explanation of class responsibilities for the classes that should be considered early:**
- **Model Layer Classes:**
- **Company:** Stores company details (ID, name, industry, contact info).
- **Student:** Stores student details (ID, name, CGPA, graduation year).
- **Drive:** Stores placement drive specifics (Drive ID, associated Company ID, role, package, date).
- **Application:** Links a Student to a Drive and tracks the application status (Applied, Shortlisted, Rejected).
- **Interview Slot:** Stores the schedule details (time, room, interviewer) for a specific shortlisted application.
- **Service Layer Classes:**
- **DriveService:** Handles business logic related to drives, such as validating drive creation dates, determining

student eligibility based on CGPA, and interacting with the Drive Dao to save or fetch records.

- (***Additional Services:*** *ApplicationService*, *CompanyService* will handle their respective entity logic).

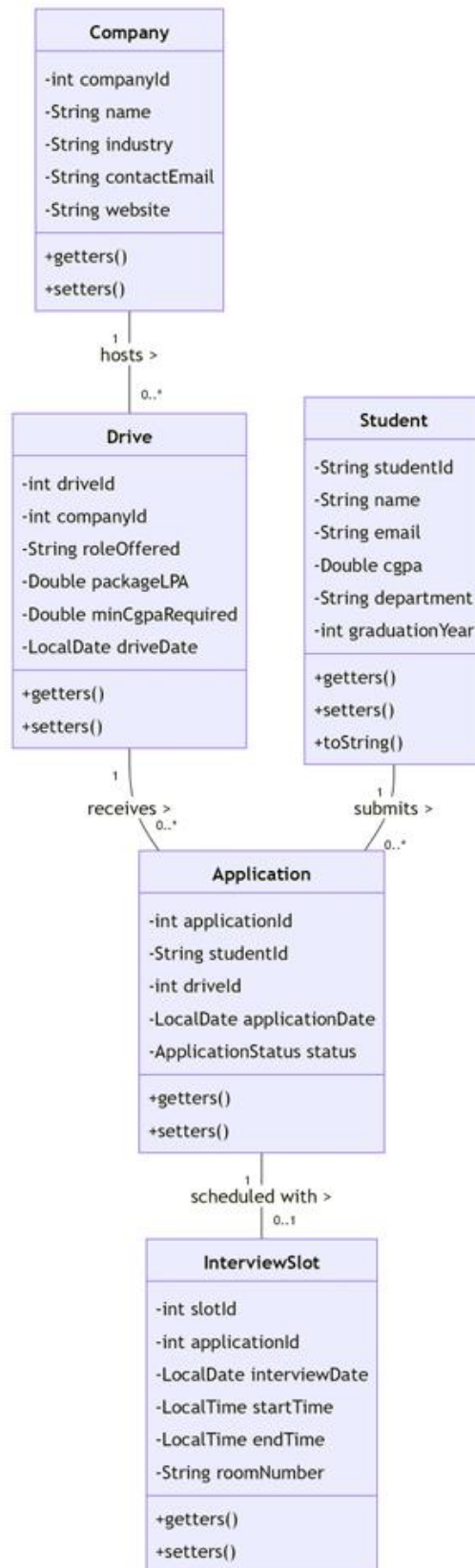
- **DAO Layer Classes:**

- **Company Dao:** Executes SQL operations specifically for the Company table in the database.
- **Application Dao:** Executes SQL operations for the Application table, such as inserting new applications or updating a status to "Shortlisted".
- (***Additional DAOs:*** *DriveDao*, *StudentDao*, *InterviewSlotDao* will handle their respective tables)

- **UML Class Relationships (For your UML Diagram):-**

- ***This section outlines the relationships we should draw in our UML Class Diagram.***
- **Company ↔ Drive:** A Company can have multiple Drives (1-to-Many).
- **Student ↔ Application:** A Student can submit multiple Applications to different drives (1-to-Many).
- **Drive ↔ Application:** A Drive can receive multiple Applications from different students (1-to-Many).
- **Application ↔ Interview Slot:** A shortlisted Application has exactly one Interview Slot (1-to-1).

- **Layer Dependency:** The ui classes depend on service classes. The service classes depend on dao classes and model classes. The dao classes depend on model classes to return database results as objects.
- **Adherence to Design Warnings:-**
- **Our architectural design strictly avoids common anti-patterns:**
- **No giant manager class:** We are not creating one giant manager class. Responsibilities are clearly divided among specific services like DriveService.
- **Clean Swing Handlers:** We strictly do not place SQL code inside Swing event handlers. Swing events will only call methods in the service layer.
- **Layered Validation:** We do not keep validation logic only in the UI. While the UI may do basic checks (e.g., empty fields), complex validations (e.g., duplicate application checks) live in the service layer.
- **Purposeful Classes:** We do not create classes that have no real responsibility. Every class mapped above is essential for the application to function.



- **Review Checklist Justification**
- **Before submitting, our design satisfies the phase checklist requirements:**
- **Clear Roles:** Does every class have a clear role? Yes, roles are strictly divided by layers (Model = Data, DAO = DB, Service = Logic, UI = Interface).
- **Sensible Relationships:** Are relationships sensible for the workflow? Yes, the relationships perfectly model the real-world flow of Companies creating Drives, and Students submitting Applications.
- **Layer Separation:** Are service and DAO responsibilities separated? Yes, Services handle business rules while DAOs strictly handle database persistence.
- **Use Case Support:** Does the UML support the use cases from Phase 1? Yes, the proposed classes and their methods will directly allow for creating drives, recording applications, shortlisting, and scheduling as outlined in Phase 1.